

Authentication Lab

Goal

The purpose of the laboratory exercise is to provide hands on experience with the development of a user authentication mechanism. Java is the official language of instruction for computer science at DTU Compute, which is why it has been chosen for this exercise.

Java Authentication

The traditional security mechanisms for the Java Virtual Machine (JVM) provide a mean to enforce access controls based on where the code was downloaded from, i.e. the code base, and who signs it. These access controls are needed because of the distributed nature of the Java platform where application may consist of packages that are dynamically downloaded from different software providers and where an applet can be downloaded over a public network and then run locally.

However, the first versions of the Java 2 platform did not provide a way to enforce similar access controls based on the principal who runs the code. To provide this type of access control, the Java 2 security architecture requires the following:

- support for authentication of principals (determining who is actually running the code)
- extensions to the existing authorization components to enforce new access controls based on the authenticated principal.

The Authentication Lab addresses the first of these problems by designing and implementing a simple authentication mechanism for a client/server application. Students who have not previously implemented an RMI application may consult the [Java RMI tutorial](#) from Oracle or consult this [short video on YouTube](#).

Exercise work:

The first task is to write a simple client/server application using RMI. The example used in this lab is a mock-up of a simple authenticated print server, such as a print server installed in a small company.

The print server must support the following operations:

- `print(String filename, String printer);` // prints file filename on the specified printer
- `queue(String printer);` // lists the print queue for a given printer on the user's display in lines of the form `!job number. !file name`.
- `topQueue(String printer, int job);` // moves job to the top of the queue

- `start();` // starts the print server
- `stop();` // stops the print server
- `restart();` // stops the print server, clears the print queue and starts the print server again
- `status(String printer);` // prints status of printer on the user's display
- `readConfig(String parameter);` // prints the value of the parameter on the print server to the user's display
- `setConfig(String parameter, String value);` // sets the parameter on the print server to value

These operations define the interface of the print server, but it is unnecessary to implement any printing capabilities for this lab, i.e. it is sufficient that the print server records the invocation of a particular operation in a logfile or prints it on the console. It must be possible to invoke all the print server operations defined in the interface from the client program.

This lab will design and implement a password-based authentication mechanism for the print server, i.e. the print server must authenticate all requests from the client. For the purpose of this lab, it is not necessary to consider enrolment of users, i.e. authentication data structures can be populated by hand. The design and implementation of the print server must, however, consider the problems of password storage, password transport, password verification and session management.

Password storage

In relation to password storage, three possible solution must be considered: passwords stored in a “system” file, passwords stored in a “public” file, where cryptography is used to ensure confidentiality and integrity of the stored data; and passwords stored in a data base management system; these options are outlined below.

- **System File:** Storing passwords in a system file relies on the operating system/file system protection mechanisms are used to ensure confidentiality and integrity of the stored data. This password file is normally not accessible to all users, so some mechanism (e.g. the SetUID mechanism in Unix) or system service is required to provide controlled access to the data stored in the file (similar to the DBMS storage described below.)
- **Public File:** Storing passwords in a public file that can be read (but not necessarily written) by all users is the traditional way to store passwords on Unix systems. Confidentiality of the passwords is normally protected by cryptographic means, whereas integrity (e.g. binding users and passwords) may either be protected by the operating system/file system protection mechanism, i.e. normal users have no write access to the file (but how are passwords then updated?) or by cryptographic means.

- **DBMS:** Storing passwords in a database (often unencrypted) relies on the security architecture, e.g. the access control mechanism, implemented in the DBMS and cryptography to provide confidentiality and integrity of the stored passwords.

Think and try how each of these three solutions may be implemented in the given context and compare and contrast the security offered by each of the proposed implementations. How the chosen solution prevents users from learning or changing the password of other users.

Password transport

You can analyse the password transport problem and discuss how to implement both individual request authentication and authenticated sessions, where the initial authentication is used to define an authenticated session, which implicitly authenticates messages exchanged. If authentication of invocation requires the transfer of any additional parameters between client and server, these should simply be added to the interface defined above.

Password verification

The password verification mechanism depends on the choice of password storage.

Session management

Signing in for every action on the print server is cumbersome. Conversely, using a print server does not require to be logged in for more than a certain amount of time. Implement a simple session management mechanism so that requests to the print server are authenticated for a period and that users get disconnected after this period of time. The program must be able to store a user session and check whether the session is still valid when making a request. If the session is not valid anymore, the user should be logged out. This exercise abstracts from the problem of the secure storage of the session and storing it in a Java object is enough. However, the generation of the session and the session lifetime must be discussed in the report.

General comments

For the purpose of this exercise, it is acceptable to assume that secure communication between client and server is ensured by other means. It is also possible to protect communications by cryptographic means; in this case, you can identify and try the relevant techniques and technologies and implement them correctly in the application.

Note: No need to submit any report or solution. This exercise is only for your practice and knowledge enhancement about authentication and practical solution about problems of password storage, password transport, password verification and session management.

Useful Links

- [Java RMI tutorial](#)
- [this short video on YouTube](#)